



LayerZero Labs Security Audit

*Stargate CreditMessagingMintableBurnable
Mechanism*

Released June 1, 2026

Performed By

Carter Snay
Oliver Willis
Ryan Laidlaw

csnay@iol.unh.edu
owillis@iol.unh.edu
rlaidlaw@iol.unh.edu



**University of
New Hampshire**
Interoperability Labs

+1-603-862-0090 | www.iol.unh.edu

Table of Contents

1	Legal Notice	2
2	Executive Summary	3
2.1	About Stargate Credit Messaging Mintable Burnable	
2.1.1	Review Timeline	
2.1.2	Scope	
3	Vulnerabilities	5
3.1	Findings Summary	
3.2	Detailed Findings	
3.2.1	Info	
	Appendix A: Our Methodology	10
	Risk Classification	
	Review Phases	
	Phase I: Initial Scoping	
	Phase II: Codebase Review	
	Phase III: Local Testing	
	Proof of Vulnerability	
	Appendix B: The Interoperability Labs	13

1 Legal Notice

The Interoperability Labs Blockchain team makes every effort to identify as many vulnerabilities in the code as possible within the given time period but assumes no responsibility for the findings presented in this document. A security audit by the team does not constitute an endorsement of the underlying business or product. The audit was time-boxed, and the review focused solely on the security aspects of the Solidity implementation of the contracts.

2 Executive Summary

The UNH Interoperability Labs conducted a security assessment for LayerZero Labs from April 9, 2026 to May 29, 2026. During this assessment, UNH Interoperability Labs reviewed LayerZero Labs code for security vulnerabilities, design issues and general weaknesses.

During the assessment, there were no vulnerabilities discovered, with 3 suggestions being identified by the team.

2.1 About Stargate Credit Messaging Mintable Burnable

Stargate is a cross-chain liquidity transport protocol that enables seamless asset transfers across multiple blockchains. In order to provide instant guaranteed finality across multiple blockchains, Stargate implements a credit allocation mechanism to immediately settle swaps locally, without the risk of reverting, rollback or double spending. Cross-chain token deliverance is then handled by the underlying protocol.

In the event when a chain is deprecated without first returning its credits to the origin chains, those credits become permanently stranded. The `Stargate CreditMessagingMintableBurnable.sol` contract was introduced to address this issue.

It serves as an emergency-only, owner-controlled mechanism to restore or correct credits through minting and burning, effectively ensuring liveness of the transfer paths for specific assets in the associated Stargate pools.

2.1.1 Review Timeline

- **April 9, 2026:** Initial audit scope review
- **April 10, 2026:** First audit report draft delivered
- **April 13, 2026:** Second report draft delivered
- **May 29, 2026:** Final report delivered

2.1.2 Scope

Project Name	LayerZero Labs		
URL	https://layerzero.network/		
Language	Solidity		
Scope	Repo	https://github.com/stargate-protocol/stargate-v2	
	Hash	f123e95d8772ccae78f20bd08141f69bb6fad98e	April 6, 2026
		7e50c9b21bc43a220187809f67d6305152a1f5df	April 10, 2026
		c2c9c7fb0a3867b590257fea6ee13ad87a24ed56	May 27, 2026

Files in Scope

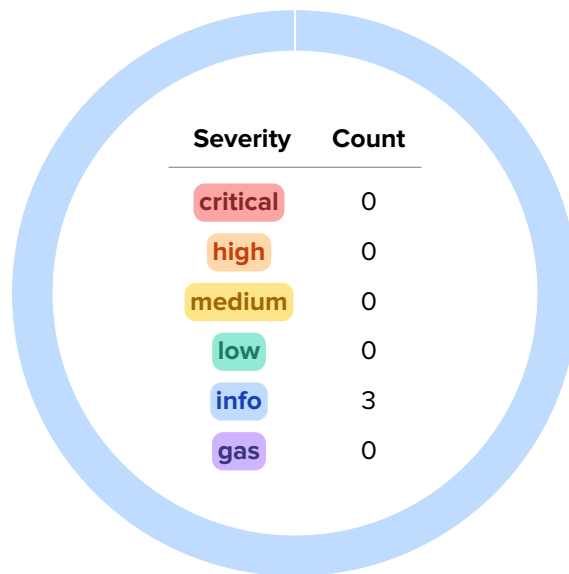
```

packages/
├── stg-evm-v2/
│   └── src/
│       ├── interfaces/
│       │   └── ICreditMessagingMintableBurnable.sol
│       └── messaging/
│           └── CreditMessagingMintableBurnable.sol

```

3 Vulnerabilities

In summary, no vulnerabilities were found, but there are 3 informational suggestions.



NOTE

The following files were renamed in commit `c2c9c7fb0a3867b590257fea6ee13ad87a24ed56` after the provided findings were reported:

- `ICreditMessagingRecovery.sol` => `ICreditMessagingMintableBurnable.sol`
- `CreditMessagingRecovery.sol` => `CreditMessagingMintableBurnable.sol`

3.1 Findings Summary

ID	Severity	Title	Status
I-01	info	Recommended Use of @inheritdoc inside CreditMessagingRecovery.sol	RESOLVED
I-02	info	Missing NatSpec Param Documentation for Event Params in ICreditMessagingRecovery.sol	RESOLVED
I-03	info	Consider A Minimum Length Check in mintCredits and burnCredits	ACKNOWLEDGED

3.2 Detailed Findings

3.2.1 Info

[I-01] Recommended Use of @inheritdoc inside CreditMessagingRecovery.sol

Category	Target
NatSpec Cleanup	CreditMessagingRecovery.sol

Description

The functions `mintCredits` and `burnCredits` inside the `CreditMessagingRecovery.sol` contract do not use the NatSpec comments made in the `ICreditMessagingRecovery` interface. By using the `@inheritdoc` NatSpec comment, documentation will remain consistent.

Recommended mitigation

Consider using `@inheritdoc` for both `CreditMessagingRecovery::mintCredits` and `CreditMessagingRecovery::burnCredits`

```

CreditMessagingRecovery.sol
diff
15 +  /// @inheritdoc ICreditMessagingRecovery
16    /// @dev Credits are applied directly on the current chain by calling receiveCredits locally,
17    ///     without deducting credits from any source or sending an LZ message.
18    function mintCredits(CreditBatch[] calldata _batches, string calldata _reason) external onlyOwner {
19
20        ...
26
27 +  /// @inheritdoc ICreditMessagingRecovery
28    /// @dev Credits are deducted on the current chain by calling sendCredits locally. No LZ message is
29    ///     ↪ sent.
30    ///     Each TargetCredit.minAmount is forwarded as minKept: the handler burns at most (
31    ///     ↪ currentCredit - minAmount)
32    ///     and returns 0 silently if currentCredit <= minAmount.
33    function burnCredits(TargetCreditBatch[] calldata _batches, string calldata _reason) external
34    ///     ↪ onlyOwner {

```

Resolution

LayerZero has accepted this suggestion, and has updated the NatSpec in the following commit:

7c50c9b21bc43a220187809f67d6305152a1f5df

[I-02] Missing NatSpec Param Documentation for Event Params in ICreditMessagingRecovery.sol

Category	Target
NatSpec Cleanup	ICreditMessagingRecovery.sol

Description

The events defined in the ICreditMessagingRecovery interface do not have NatSpec descriptions for the event parameters.

Recommended mitigation

We suggest including @param NatSpec descriptions for the CreditsMinted and CreditsBurned events.

```

ICreditMessagingRecovery.sol
diff
12 interface ICreditMessagingRecovery {
13     /// @notice Thrown when mintCredits or burnCredits is called with an empty reason string.
14     error CreditMessagingRecovery_EmptyReason();
15
16     /// @notice Emitted by mintCredits function with all batches that were applied.
17 +    /// @param batches The list of credit batches that were minted.
18 +    /// @param reason The reason for the minting.
19     event CreditsMinted(CreditBatch[] batches, string reason);
20
21     /// @notice Emitted by burnCredits function with the actual credits removed by the handler.
22 +    @param batches The list of credit batches that were burned.
23 +    @param reason The reason for the burning.
24     event CreditsBurned(CreditBatch[] batches, string reason);

```

Resolution

LayerZero has accepted this suggestion, and has updated the NatSpec in the following commit:

7c50c9b21bc43a220187809f67d6305152a1f5df

[I-03] Consider A Minimum Length Check in `mintCredits` and `burnCredits`

Category	Target
Business Logic	<code>CreditMessagingRecovery.sol</code>

Description

In the functions `mintCredits` and `burnCredits`, the validation check on `bytes(_reason).length == 0` is intended to ensure the `_reason` parameter is a non-empty string. However, this doesn't encapsulate conditions like using a space, or other non-standard characters.

Recommended mitigation

We suggest adding a minimum length greater than 1 to validate on in order to prevent passing a single character into the `_reason` parameter.

Resolution

LayerZero has acknowledged this, by stating that the responsibility to provide a meaningful message is on the signer and not on the contract.

Appendix A: Our Methodology

The UNH Interoperability Labs audit team follows a comprehensive methodology in ensuring the security and reliability of smart contracts and Web3 protocols. While the specific testing procedures performed vary between the project and protocol, the tooling and manual review process remains the same to ensure thorough analysis has been completed on all items within the defined scope of the audit. Throughout the security review, the audit team maintains communication with the development team, providing feedback on identified vulnerabilities and optimizations. The following sections provide an overview of our systematic audit process and methodology.

Risk Classification

			Impact →			
Likelihood ↓		Informational	Low	Medium	High	Critical
	Very Unlikely	Info	Low	Low	Medium	Critical
	Unlikely	Info	Low	Low	Medium	Critical
	Possible	Info	Low	Medium	High	Critical
	Likely	Info	Low	Medium	High	Critical
	Very Likely	Low	Medium	High	Critical	Critical

We use the PricewaterhouseCoopers-style matrix to provide comprehensive risk assessment. See the documentation for more details.

Review Phases

Phase I: Initial Scoping

- Independent review of project documentation to understand the business logic of the project.
- Identification of critical components and key areas of focus and possible areas of exploitation.
- Ensure that the project's documentation is accurate, complete, understandable, and there is alignment between the code and the documentation.
- Discussion with the development team to clarify objectives, expectations, and known issues.

Phase II: Codebase Review

- **Static Analysis:** Automated tools to scan for common vulnerabilities with Slither and Aderyn.
- **Manual Review:** In-depth inspection of the code by the audit team to identify issues, including unsafe coding practices from known previous exploits.
- **Function State Machine Diagramming:** Generate a flow diagram illustrating the intended transaction paths, as well as unintended and potentially exploitable paths.

Phase III: Local Testing

Methods:

- **Unit Testing:** Validate individual functions for correctness.
- **Integration Testing:** Ensure that different components interact as expected.

Techniques Used in This Audit:

- **Unit Testing**
- **Manual Review**
- **Function State Machine Diagramming**

Proof of Vulnerability

Objective: Prove how a found exploit can be executed.

Activities:

- Perform controlled attacks on a local fork of the protocol to show how an exploit can be executed.
- Test edge cases and unexpected scenarios discovered in the diagramming phase.

By following a structured and comprehensive methodology, we aim to provide actionable insights to strengthen the security and reliability of the protocol. This ensures security, resilience, and long-term success for the protocol.

Appendix B: The Interoperability Labs

The University of New Hampshire Interoperability Labs (UNH-IOL) is the foremost independent testing facility for data networking companies worldwide.

We accelerate the launch of innovative products by providing standards and compliance testing to ensure that devices meet industry standards. Whether it's traditional Ethernet or advanced technologies like 5G, blockchain, and autonomous vehicles, our services provide comprehensive testing for device interoperability, conformance, and certification.

Our state-of-the-art laboratory and 36 years of extensive experience make it a strategic resource for industry startups and Fortune 500 companies needing collaboration, innovation, and standards development to help them shape the future of networking.

Join us and become a part of a community driving the next generation of networking technology. Together, we can shape the future of networking.

<https://www.iol.unh.edu/membership>

University of New Hampshire Interoperability Labs

21 Madbury Rd., Ste 100 Durham, NH 03824-4716

+1-603-862-0090 | www.iol.unh.edu

Contact our Blockchain Team: blockchain@iol.unh.edu